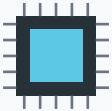
Computer Architecture • Chapter 3

Case Study: the Intel 8086



registers, segments, instructions, stack, procedures, interrupts

Teacher: Dr. Benabderrezak Youcef

Course: UEF31, Architecture des ordinateurs

Goals of this chapter

At the end, you can:

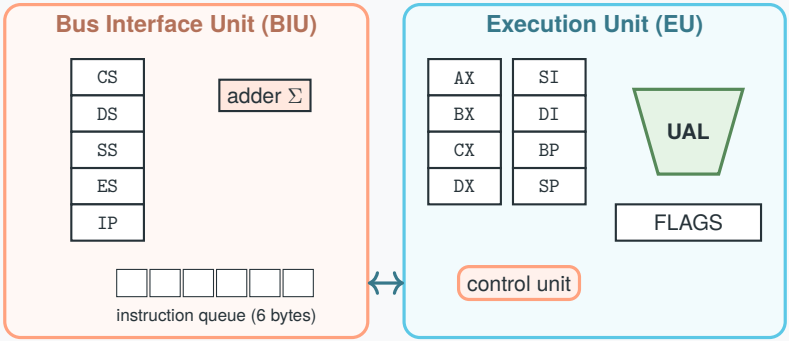
- use the 8086 registers and compute physical addresses;
- write and trace programs with transfers, arithmetic, logic, jumps and loops;
- use the stack, procedures and interrupts.

The 80x86 family

Processor	Year	Bits	Memory	New
8086	1978	16	1 MB	first of the family
80286	1982	16	16 MB	protected mode
80386	1985	32	4 GB	IA-32
Athlon 64	2003	64	huge	AMD64
Core i7	2008	64	huge	several cores

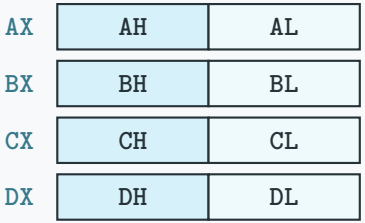
Backward compatibility: a modern Core i7 still runs 8086 programs.

Inside the 8086: two units working together



BIU: talks with memory and fetches in advance. EU: decodes and runs.

The general registers



If AX = 1234_H: AH = 12_H, AL = 34_H.

Pointers, segments, IP and FLAGS

Reg.	Job
SI, DI	source / destination index
SP	top of the stack
BP	data inside the stack
IP	next instruction

Segment	Start of…
CS	the code
DS	the data
SS	the stack
ES	an extra segment



Segmented addressing

- Registers are 16 bits (64 KB), but the address bus is 20 bits (1 MB).

- So an address uses **two** registers: `segment:offset`.

The rule

$$\text{physical address} = \text{segment} \times 16 + \text{offset}$$

Analogy

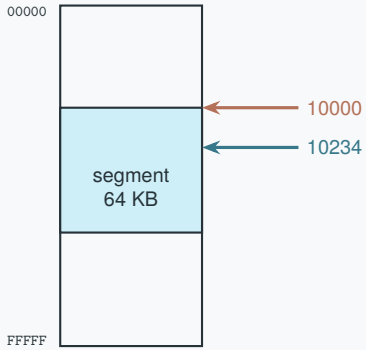
Segment = street number, offset = house number in the street.

Example: CS = 1000H, IP = 0234H

Step by step

1. $1000_H \times 16 = 10000_H$.
2. $10000_H + 0234_H = 10234_H$.

1023:0004 gives the same address!



Default pairs and addressing modes

Offset	Segment
IP	CS
SP, BP	SS
BX, SI, direct	DS
DI (strings)	ES

Mode	EA
`[0200h]`	0200_H
`[bx]`	BX
`[bx+4]`	BX + 4
`[bx+si+4]`	BX + SI + 4



`[bx+si+4]` with DS = 2000, BX = 0100, SI = 0020

EA $= 0124_H$, physical address $= 20000_H + 0124_H = 20124_H$.

A first program

```
1 ; My first 8086 program: print a message on the screen
2 org 100h
3
4     lea dx, msg
5     mov ah, 09h
6     int 21h
7     ret
8
9 msg db 'Salam, I am the 8086!$'
```

```
Salam, I am the 8086!
```

In a .COM program, CS = DS = SS = ES, and the code starts at 100_H (`org 100h`).

The program in memory

Offset	Bytes	Content
0100	`8D 16 09 01`	`lea dx, msg`
0104	`B4 09`	`mov ah, 09h`
0106	`CD 21`	`int 21h`
0108	`C3`	`ret`
0109	`53 61 6C 61 6D …`	“Salam…”

Watch out

Data goes **after** `ret`; otherwise the processor runs the letters as instructions.

How `mov cx, [si]` runs

Step	Unit	What happens
1	BIU	fetches the bytes 8B 0C into the queue
2	EU	decodes: “word from memory to CX, address in SI”
3	EU	EA = SI = 0112_H
4–5	BIU	sends DS $\times 16 + 0112_H$, gets $ABCD_H$
6	EU	CX = $ABCD_H$; the flags do not change

Transfer: `mov`, `xchg`, `lea`

```
1  ; Data transfer: MOV, XCHG, LEA
2  org 100h
3
4      mov ax, 1234h
5      mov bl, ah
6      mov bh, al
7      xchg ax, bx
8      lea si, num
9      mov cx, [si]
10     mov [res], ax
11     ret
12
13 num dw 0ABCDh
14 res dw 0
```

Line	AX	BX
4	1234	0000
6	1234	3412
7	3412	1234

Line 8:

SI = 0112 (address).
Line 9: CX = ABCD (content).

Arithmetic

```
1  ; Arithmetic: ADD, SUB, INC, DEC, NEG
2  org 100h
3
4      mov al, 25
5      add al, 17
6      sub al, 2
7      inc al
8      dec al
9      neg al
10     mov bx, 1000
11     add bx, 2500
12     ret
```

AL: 25, 42, 40, 41, 40, then **D8$_H$** $= -40$.



-40

$0010\,1000 \rightarrow 1101\,0111 \rightarrow +1 =$ D8$_H$

Multiply and divide

Instruction	Operation	Result
`mul bl`	AL $\times$ BL	AX
`mul cx`	AX $\times$ CX	DX:AX
`div bl`	AX $\div$ BL	AL = quotient, AH = remainder

Real results: 12×11 gives AX = 0084_{H}; 300×500 gives DX:AX = 0002:49F0 (150000); $47 \div 5$ gives AX = 0209_{H} (quotient 9, remainder 2).

Adding 32-bit numbers: `adc`

```
; Add two 32-bit numbers with a 16-bit processor
; 0001FFFFh + 00000001h = 00020000h
org 100h

    mov ax, 0FFFFh
    mov dx, 0001h
    add ax, 0001h
    adc dx, 0000h
    ret
```

The carry of the low words (CF = 1) is added by `adc`: DX:AX = 0002:0000.

Logic and shifts

```
1  ; Logic and shift instructions
2  org 100h
3
4      mov al, 10110110b
5      and al, 00001111b
6      or  al, 10000000b
7      xor al, 11111111b
8      not al
9      mov bl, 5
10     shl bl, 1
11     shl bl, 1
12     shr bl, 1
13     xor cx, cx
14     ret
```

- `and`: switch bits off
- `or`: switch bits on
- `xor`: flip bits
- `shl`/`shr`: $\times 2$ / $\div 2$

BL: 5, 10, 20, 10.

Jumps

Instruction	Jumps if…
`jmp`	always
`je`, `jne`	equal, not equal (ZF)
`jg`, `jl`	greater, less (**signed**)
`ja`, `jb`	above, below (**unsigned**)

`cmp a, b` computes $a - b$ and sets the flags; the jump reads them. For the same bits $C8_H$: `ja` sees $200 > 100$, but `jg` sees $-56 < 100$.

If … else

```
1  ; IF ... ELSE : print which number is bigger
2  org 100h
3
4      mov al, [a]
5      cmp al, [b]
6      jg  first
7      lea dx, msg2
8      jmp show
9  first:
10     lea dx, msg1
11 show:
12     mov ah, 09h
13     int 21h
14     ret
15
16 a    db 7
17 b    db 12
18 msg1 db 'a is bigger$'
19 msg2 db 'b is bigger or equal$'
```

```
b is bigger or equal
```

- Line 5: $7 - 12 < 0$.
- Line 6: no jump.
- Line 8: `jmp show` is needed, or we also run line 10.

Loops with `loop`

```
1  ; FOR loop with LOOP: sum = 1 + 2 + ... + 10
2  org 100h
3
4      mov ax, 0
5      mov cx, 10
6  again:
7      add ax, cx
8      loop again
9      mov [sum], ax
10     ret
11
12 sum dw 0
```

`loop` = `dec cx`, then jump if CX $\neq 0$.

CX	AX after add
10	10
9	19
8	27
…	…
1	**55**

Machine code

Bytes	Instruction
B8 34 12	`mov ax, 1234h`
89 C3	`mov bx, ax`
04 05	`add al, 5`
88 07	`mov [bx], al`
EB 01	`jmp fin`

89 C3

89 = 100010 0 1: `mov`, d = 0, w = 1 (16 bits).
C3 = 11 000 011: mod = register, reg = AX, r/m = BX.

EB 01: jump 1 byte forward (relative).

Input and output with DOS

AH	`int 21h`
01	read a key into AL
02	print DL
09	print a $ string (DX)
4C	end the program

Ports:

`in al, 60h`, `out 61h, al`.

```
 7      mov ah, 01h
 8      int 21h
 9      sub al, 32
10      mov bl, al
11      lea dx, ans
12      mov ah, 09h
13      int 21h
14      mov dl, bl
15      mov ah, 02h
16      int 21h
17      ret
```

```
Type a letter: m
Uppercase: M
```

The stack

- LIFO: **last in, first out**, like a pile of plates.
- `push`: SP ← SP - 2, then write. `pop`: read, then SP ← SP + 2.



After `pop ax` and `pop bx`, AX and BX have swapped (real run).

Procedures: `call` and `ret`

```
10  print_num proc
11      mov bx, 10
12      mov cx, 0
13  divide:
14      mov dx, 0
15      div bx
16      push dx
17      inc cx
18      cmp ax, 0
19      jne divide
20  show:
21      pop dx
22      add dl, '0'
23      mov ah, 02h
24      int 21h
25      loop show
26      mov dl, ' '
27      int 21h
28      ret
29  print_num endp
```

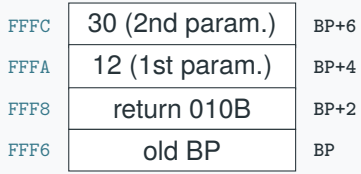
- `call` pushes the return address.
- `ret` pops it into IP.
- The stack reverses the digits 6, 2, 0, 2.

```
2026 7
```

Parameters on the stack

```
12 addtwo proc
13     push bp
14     mov bp, sp
15     mov ax, [bp+4]
16     add ax, [bp+6]
17     pop bp
18     ret 4
19 addtwo endp
```

`ret 4` also removes the two parameters.



$12 + 30 = 42$ in AX.

Interrupts

- Kinds: hardware, software, exceptions.
- The address of handler *n* is at $4n$.
- `int n` saves FLAGS, CS, IP; `iret` comes back.

```
 4      mov ax, 0
 5      mov es, ax
 6      lea ax, handler
 7      mov es:[60h*4], ax
 8      mov es:[60h*4+2], cs
 9      mov bl, 'A'
10      int 60h
11      int 60h
12      ret
13
14  handler:
15      mov dl, bl
16      mov ah, 02h
17      int 21h
18      inc bl
19      iret
```

```
AB
```

Summary

Remember

- physical address $=$ segment $\times\ 16 +$ offset.
- `cmp` sets the flags; jumps read them (signed: `jg`, unsigned: `ja`).
- The stack (LIFO) keeps return addresses, registers and parameters.